

Custom Arithmetic Library

Project Report

Thamatam Jayadev Reddy

May 2, 2025

Contents

1	Introduction	2
2	Git Commit Snapshot	3
3	Core Logic behind implementation	3
4	Limitations	4
5	Verification Approach	4
6	Key Learnings	4

1 Introduction

Our library consists of two primary classes: `AInteger` and `AFloat`, both of which encapsulate custom arithmetic operations like addition, subtraction, multiplication, and division.

Class Diagram (UML)

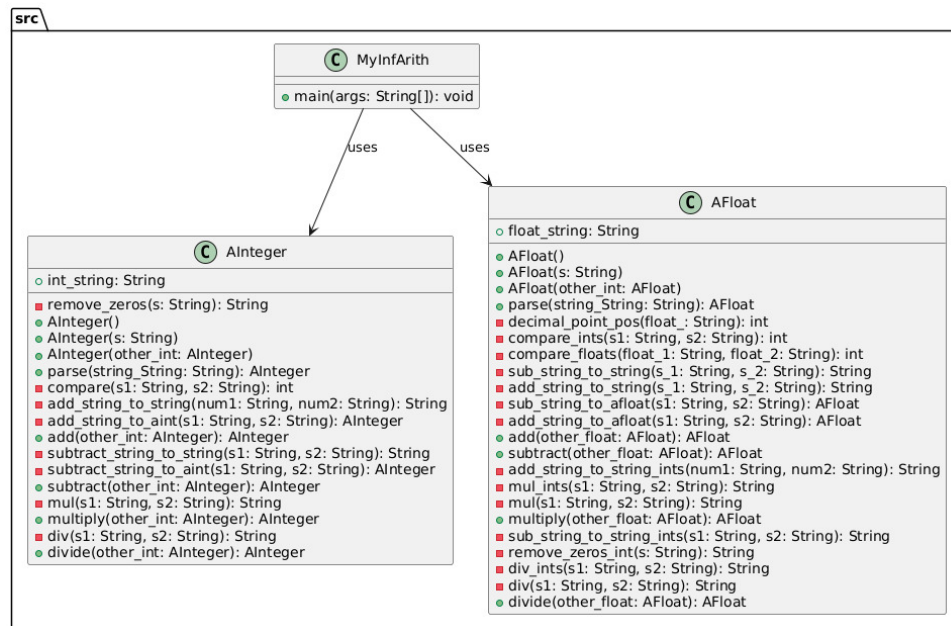


Figure 1: Class diagram for `AIInteger` and `AFloat`

2 Git Commit Snapshot

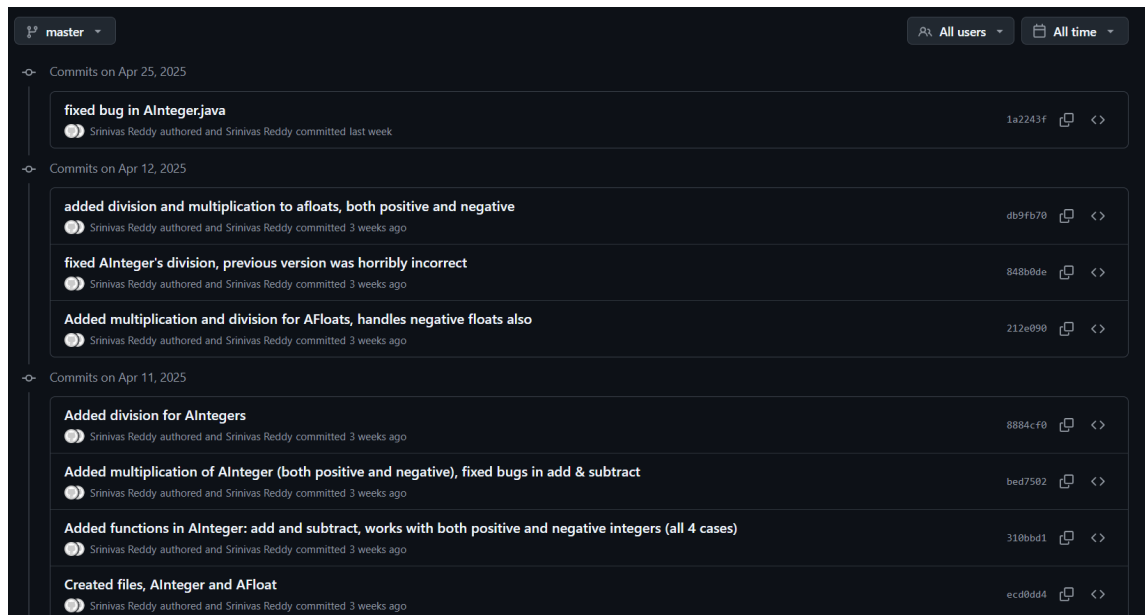


Figure 2: Snapshot of git commits

3 Core Logic behind implementation

Core Logic of AFloat

- Parses input strings to separate integer and decimal parts.
- Aligns decimal places by padding with zeros where needed.
- Performs digit-by-digit arithmetic (e.g., long addition or multiplication).
- Uses custom carry/borrow logic for correct computation.
- Combines results and formats them to maintain floating-point form.
- **Core Logic: AInteger**
- **Addition & Subtraction:** Performed digit-by-digit from least to most significant digit, with careful carry/borrow handling.
- **Multiplication:** Simulates manual multiplication (schoolbook method) and accumulates partial products.
- **Division:** Implements long division by subtracting multiples of the divisor from the dividend using iterative comparisons.
- **Comparison:** Length-based and lexicographic comparison functions help determine ordering for operations.

4 Limitations

- cases like division by 0.00 and 0.000 are not yet handled
- Floating-point accuracy depends on internal parsing, which may vary.

5 Verification Approach

- Unit tests were written for all arithmetic operations.
- Manual tests were done using large numbers and edge cases.

6 Key Learnings

- Deepened understanding of object-oriented design.
- Learned how to implement and test custom numerical classes.
- Gained hands-on experience with Git, version control workflows, and LaTeX documentation.